

# SYSTEM DESIGN

## INTERVIEW MASTER GUIDE

*Placements 2026 — India Edition*

Product Companies · Fintech · Service-Based · Unicorns

*Fundamentals* • *Building Blocks* • *Hot Case Studies* • *Rapid Revision*

## How To Use This Guide

This guide is built for HLD (High-Level Design) rounds at Indian product companies (Flipkart, Swiggy, Zomato, PhonePe, Razorpay, Ola, Meesho, Cred, Amazon India), FAANG India offices, and system-design-flavoured HLD/LLD rounds at service-based firms (TCS Digital, Infosys, Cognizant, Wipro) and fintech unicorns.

- Part 1 – The Framework: a repeatable 7-step method to answer ANY design question, even ones you've never seen.
- Part 2 – Core Fundamentals: the concepts every single design question draws from (caching, sharding, queues, CAP, etc.) — memorize this once, reuse everywhere.
- Part 3 – Reusable Building Blocks: mini-designs (rate limiter, ID generator, notification service) that get reused as sub-components inside bigger systems.
- Part 4 – Hot Case Studies: full walkthroughs of the questions actually being asked in 2025–26 — Zomato/Swiggy, Uber/Ola, WhatsApp, Instagram, Twitter/X, Netflix/Hotstar, BookMyShow, UPI/Payment Gateway, Google Drive/Dropbox, and E-commerce (Flipkart-style).
- Part 5 – India-Specific Interview Patterns: what each type of company actually probes for.
- Part 6 – One-Page Rapid Revision: the night-before-the-interview cheat sheet.

### Study Strategy

- Read Parts 1–2 fully first — they are the vocabulary everything else is written in.
- For Part 4, don't memorize architectures verbatim. Memorize the REASONING for each design choice — interviewers probe 'why', not 'what'.
- Re-derive one case study from memory (no notes) 48 hours before your interview — that's the real test of retention.
- Numbers matter: always be ready to justify capacity estimates with rough math on the spot.

## Part 1: The Universal Framework

Every system design interview — whether it's 'Design Zomato' or 'Design a parking lot' — can be answered with the same 7-step structure. Interviewers are grading your PROCESS, not whether you reproduce a known architecture.

### The RESHAPE Method

| Step                     | What You Do                                                                                | Time (~45 min interview) |
|--------------------------|--------------------------------------------------------------------------------------------|--------------------------|
| R – Requirements         | Clarify functional + non-functional requirements. Ask questions before designing anything. | 5 min                    |
| E – Estimation           | Back-of-envelope: users, QPS, storage, bandwidth.                                          | 5 min                    |
| S – Storage / Data Model | Define entities, relationships, choose SQL vs NoSQL.                                       | 5 min                    |
| H – High-Level Design    | Draw boxes: client, LB, services, DB, cache, queue. Explain data flow.                     | 10 min                   |

| Step                    | What You Do                                                     | Time (~45 min interview) |
|-------------------------|-----------------------------------------------------------------|--------------------------|
| A – API Design          | Define 3–5 key REST/gRPC endpoints with request/response shape. | 5 min                    |
| P – Pick a Deep Dive    | Interviewer picks (or you propose) ONE hard part to zoom into.  | 10 min                   |
| E – Evaluate Trade-offs | Bottlenecks, failure modes, scaling further, what you'd change. | 5 min                    |

## Step 1: Requirements — The Questions To Always Ask

### Functional Requirements

- What are the core use-cases? (List 3–5, no more — scope control is a skill being tested.)
- Who are the actors/users? (customer, driver, restaurant, admin...)
- Explicitly state what's OUT of scope. E.g. "I will design the order + delivery flow, not the payment gateway internals."

### Non-Functional Requirements (NFRs) — always cover these 5

- **Scale:** how many users (DAU/MAU), how many requests/sec at peak?
- **Availability vs Consistency:** can we tolerate stale data for higher uptime? (CAP theorem)
- **Latency:** what response time is acceptable? (chat: <100ms, feed: <500ms, batch: seconds is fine)
- **Durability:** can we ever lose data? (payments: never. cache: fine to lose.)
- **Read vs Write ratio:** read-heavy (Twitter feed, 100:1) needs caching+replicas; write-heavy (logging, IoT) needs sharding+queues.

#### Interview Tip

- Say your NFR priorities OUT LOUD and justify them: "Since this is a payments system, I'll prioritize Consistency and Durability over Availability." This single sentence signals strong seniority.
- Never skip requirements gathering even under time pressure — designing the wrong thing fast is worse than designing the right thing slowly.

## Step 2: Back-of-Envelope Estimation

You don't need to be exact — you need to reason in orders of magnitude, fast.

| Quantity to know by heart   | Value                                                                                 |
|-----------------------------|---------------------------------------------------------------------------------------|
| 1 day                       | $\approx 86,400 \text{ sec} \approx 10^5 \text{ sec}$ (round number to memorize)      |
| Seconds in a month          | $\approx 2.6 \times 10^6$                                                             |
| 1 KB                        | $10^3 \text{ bytes}$ ; 1 MB = $10^6$ ; 1 GB = $10^9$ ; 1 TB = $10^{12}$               |
| QPS from DAU                | $\text{QPS} \approx (\text{DAU} \times \text{actions/user/day}) / 86,400$             |
| Peak traffic factor         | Peak QPS $\approx 2\text{--}3 \times$ average QPS                                     |
| Read replicas rule of thumb | 1 primary handles $\sim 5\text{--}10\text{K}$ simple writes/sec before you must shard |

## Latency Numbers Every Engineer Should Know (approx, memorize the RELATIVE gaps)

| Operation                                   | Approx Latency                      |
|---------------------------------------------|-------------------------------------|
| L1/L2 cache reference                       | ~1 ns                               |
| RAM access                                  | ~100 ns                             |
| SSD random read                             | ~100 $\mu$ s (150–300 $\times$ RAM) |
| Round trip within same data center          | ~0.5 ms                             |
| Redis / in-memory cache GET                 | ~1 ms                               |
| Disk seek (HDD)                             | ~10 ms                              |
| DB query over network (simple)              | ~5–10 ms                            |
| Round trip cross-continent (e.g. Mumbai–US) | ~150 ms                             |

### Exam-Focus / Interview Focus

- The point of these numbers isn't precision — it's to justify design decisions: "Cache is ~100 $\times$  faster than DB, so for a read-heavy feed I'll cache aggressively."
- If asked 'estimate storage for 5 years', just do: daily volume  $\times$  365  $\times$  5, round generously, then say 'and I'd add replication factor 3 on top for durability.'

## Part 2: Core Fundamentals (Memorize Once, Use Everywhere)

Every case study in Part 4 is just a remix of these ~18 concepts. Master this section and you can improvise any design question.

### 2.1 Scalability: Vertical vs Horizontal

| Vertical Scaling (Scale Up)                 | Horizontal Scaling (Scale Out)            |
|---------------------------------------------|-------------------------------------------|
| Add more CPU/RAM to one machine             | Add more machines                         |
| Simple, no distributed-systems complexity   | Complex — needs LB, data partitioning     |
| Hard ceiling (biggest machine available)    | Near-limitless, but adds network overhead |
| Single point of failure                     | Fault tolerant if done right              |
| Good for: relational DB primary, small apps | Good for: stateless app servers, web tier |

### 2.2 CAP Theorem & PACELC

In a distributed system, during a network Partition, you must choose between Consistency and Availability. You cannot have all three of C, A, P at once (P is not optional — partitions WILL happen).

- **CP systems:** reject requests rather than return stale data. E.g. MongoDB (config), HBase, Zookeeper, bank ledgers.
- **AP systems:** always respond, possibly with stale data, then reconcile later. E.g. Cassandra, DynamoDB, DNS.

- **PACELC extension:** even without a Partition (E = Else), you trade Latency vs Consistency. This is the more practically useful lens — use it in interviews to sound advanced.

#### Interview Tip

- For 'Design WhatsApp/Zomato order status' → favor Availability (AP) — a slightly stale delivery status is fine, an app that's down is not.
- For 'Design a payment/wallet system' → favor Consistency (CP) — never show wrong balance, even if it means a spinner for 2 extra seconds.

## 2.3 ACID vs BASE

| ACID (SQL / RDBMS)                              | BASE (NoSQL, distributed)                              |
|-------------------------------------------------|--------------------------------------------------------|
| Atomicity, Consistency, Isolation, Durability   | Basically Available, Soft state, Eventually consistent |
| Strong guarantees, harder to scale horizontally | Loose guarantees, scales horizontally with ease        |
| Use for: payments, orders, inventory counts     | Use for: feeds, view counts, presence/status, logs     |

## 2.4 SQL vs NoSQL — Choosing The Right Database

| Type             | Example                    | Best For                                                                               |
|------------------|----------------------------|----------------------------------------------------------------------------------------|
| Relational (SQL) | PostgreSQL, MySQL          | Structured data, transactions, joins, strong consistency (orders, payments, inventory) |
| Key-Value        | Redis, DynamoDB            | Caching, sessions, leaderboard, feature flags — O(1) lookups by key                    |
| Document         | MongoDB, Couchbase         | Semi-structured/nested data, flexible schema (product catalogs, user profiles)         |
| Wide-Column      | Cassandra, HBase, Bigtable | Massive write throughput, time-series, chat messages, IoT telemetry                    |
| Graph            | Neo4j                      | Highly connected data — social graphs, recommendation engines                          |
| Search Index     | Elasticsearch, Solr        | Full-text search, autocomplete, log analytics                                          |
| Time-Series      | InfluxDB, TimescaleDB      | Metrics, monitoring, sensor data                                                       |

#### Golden Rule

- If you need ACID transactions across entities (money, inventory) → SQL.
- If you need horizontal write scale + flexible schema + eventual consistency is OK → NoSQL.
- Real systems mix both: e.g. Zomato uses SQL for orders/payments, Redis for cart/session, Cassandra for location pings, Elasticsearch for restaurant search.

## 2.5 Indexing

- An index is a separate data structure (usually a B-Tree or B+ Tree, sometimes a Hash index) that lets you find rows without scanning the whole table.
- Trade-off: speeds up reads ( $O(\log n)$  instead of  $O(n)$ ) but slows down writes (index must be updated) and uses extra storage.
- Composite indexes: order of columns matters — index (user\_id, created\_at) helps queries filtering by user\_id, or by user\_id+created\_at, but NOT by created\_at alone.
- Full-text search doesn't fit a B-Tree well — that's why we bolt on Elasticsearch (inverted index) for search-heavy features.

## 2.6 Database Replication

| Pattern                        | How It Works                                                                         | Trade-off                                                            |
|--------------------------------|--------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Leader-Follower (Master-Slave) | Writes go to 1 leader; reads can be served by N followers (async or sync replicated) | Simple; follower reads can be stale (replication lag)                |
| Multi-Leader                   | Multiple nodes accept writes, sync between them                                      | Better write availability across regions; conflict resolution needed |
| Leaderless (Quorum-based)      | Any node accepts read/write; use $W+R > N$ quorum for consistency                    | High availability; used by Cassandra/DynamoDB                        |

## 2.7 Database Sharding (Horizontal Partitioning)

Splitting one big table across multiple DB instances ("shards") so no single machine holds everything.

- **Range-based sharding:** split by key ranges (e.g. user\_id 1–1M, 1M–2M). Simple, but risk of hotspots.
- **Hash-based sharding:**  $\text{hash}(\text{key}) \% N$  shards. Even distribution, but resharding is painful — solved by consistent hashing (see 2.9).
- **Geo/Directory-based sharding:** shard by region (India-West, India-South) — also helps with data-locality laws.
- **Challenges introduced:** cross-shard joins are hard, need a lookup/directory service, resharding is expensive, transactions spanning shards need 2PC or Sagas.

## 2.8 Caching Strategies

| Strategy                  | How It Works                                                    | Risk                                                                   |
|---------------------------|-----------------------------------------------------------------|------------------------------------------------------------------------|
| Cache-Aside (Lazy load)   | App checks cache first; on miss, reads DB, then populates cache | Most common. First request after eviction is slow (cache miss penalty) |
| Write-Through             | Write goes to cache AND DB synchronously                        | Consistent, but write latency = cache + DB write                       |
| Write-Back (Write-Behind) | Write goes to cache only; flushed to DB asynchronously later    | Fast writes, but risk of data loss if cache crashes before flush       |
| Write-Around              | Write goes directly to DB, bypassing cache                      | Good for data rarely re-read right after write                         |

- **Eviction policies:** LRU (Least Recently Used) — most common; LFU (Least Frequently Used); FIFO; TTL-based expiry.
- **Cache invalidation:** the classic hard problem. Options: TTL expiry, explicit invalidation on write, versioned cache keys.
- **Redis vs Memcached:** Redis supports rich data structures (sorted sets, lists, pub-sub), persistence, replication — use it by default. Memcached is simpler, pure key-value, multi-threaded.
- **CDN:** caches static content (images, videos, JS/CSS) at edge locations near the user — cuts latency + origin load. Critical for Netflix/Hotstar, Instagram images.

#### Interview Tip

- Whenever you mention 'cache', immediately state the invalidation strategy — interviewers pounce on 'stale cache' scenarios. Have an answer ready: TTL + write-through invalidation is a safe default to quote.

## 2.9 Consistent Hashing

Solves the resharding problem: when you add/remove a cache or DB node, naive  $\text{hash}(\text{key}) \% N$  remakes almost ALL mappings. Consistent hashing arranges nodes on a hash ring; adding/removing a node only remaps the keys owned by that node's neighbors ( $\sim 1/N$  of keys), not everything.

- Virtual nodes (vnodes): each physical node gets multiple points on the ring to ensure even load distribution.
- Used in: Cassandra, DynamoDB, Redis Cluster, CDN request routing, load balancers, distributed caches.

## 2.10 Load Balancing

| Layer            | What It Balances On                               | Example               |
|------------------|---------------------------------------------------|-----------------------|
| L4 (Transport)   | IP + port, fast, no content awareness             | AWS NLB, IPVS         |
| L7 (Application) | HTTP headers, URL path, cookies — smarter routing | NGINX, AWS ALB, Envoy |

- **Algorithms:** Round Robin, Weighted Round Robin, Least Connections, IP Hash (sticky sessions), Consistent Hashing.
- **Health checks:** LB pings /health endpoint periodically; removes dead nodes from rotation automatically.

## 2.11 Message Queues & Event Streaming

| Tool                 | Model                                                       | Best For                                                                          |
|----------------------|-------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Kafka                | Distributed log, pub-sub, replayable, partitioned           | High-throughput event streaming, order events, analytics pipelines, activity logs |
| RabbitMQ             | Traditional message broker, complex routing (exchanges)     | Task queues, RPC-style messaging, priority queues                                 |
| AWS SQS              | Managed simple queue, at-least-once delivery                | Decoupling microservices, simple job queues                                       |
| Pub/Sub (Redis, SNS) | Fire-and-forget broadcast, no persistence guarantee (Redis) | Real-time notifications, chat presence broadcast                                  |

- **Why use a queue:** decouples producer/consumer speed, absorbs traffic spikes, enables async processing (send email, push notification, generate invoice), improves fault tolerance (retry on failure).
- **Delivery guarantees:** at-most-once (may lose), at-least-once (may duplicate — need idempotency), exactly-once (expensive, rarely truly needed).

## 2.12 Idempotency

An operation is idempotent if calling it multiple times has the same effect as calling it once. Critical whenever clients retry (flaky mobile networks are the norm in India!).

- Pattern: client generates a unique Idempotency-Key (UUID) per logical request; server stores processed keys and returns the cached result on duplicate submission.
- Essential for: payment APIs, order placement, "Pay Now" double-tap protection — a must-mention in any payments/e-commerce design.

## 2.13 Rate Limiting

| Algorithm              | How It Works                                                  | Note                                                                     |
|------------------------|---------------------------------------------------------------|--------------------------------------------------------------------------|
| Fixed Window Counter   | Count requests in fixed time buckets (e.g. per minute)        | Simple, but allows 2x burst at window boundary                           |
| Sliding Window Log     | Store timestamp of every request, count within rolling window | Accurate but memory-heavy                                                |
| Sliding Window Counter | Weighted average of current + previous window                 | Good accuracy/memory trade-off — most used in practice                   |
| Token Bucket           | Bucket refills at fixed rate; request consumes a token        | Allows bursts up to bucket size; industry favorite (used by Stripe, AWS) |
| Leaky Bucket           | Requests queued and processed at constant rate                | Smooths bursts completely, adds queuing delay                            |

## 2.14 Proxy, API Gateway & Microservices

- **Forward proxy:** sits in front of clients (hides client identity from server, e.g. corporate proxy).
- **Reverse proxy:** sits in front of servers (hides server identity from client, does LB/SSL termination/caching) — e.g. NGINX.
- **API Gateway:** single entry point for microservices — handles auth, rate limiting, routing, request aggregation, protocol translation.
- **Monolith vs Microservices:** monolith = simpler to build/deploy/debug, but scales as one unit and one bug can crash everything. Microservices = independent scaling/deployment/tech choices, but adds network overhead, distributed tracing complexity, and eventual consistency across services.

### Interview Tip

- Don't default to "microservices are always better" — a mature answer is: "I'd start with a modular monolith and split out services (like payments, search) only when their scaling or team-ownership needs diverge." This is what senior interviewers want to hear.

## 2.15 Unique ID Generation in Distributed Systems

- **Why not auto-increment:** doesn't work across multiple DB shards without coordination.
- **UUID:** 128-bit, globally unique, no coordination needed, BUT not sortable by time and larger index size.
- **Snowflake (Twitter) ID:** 64-bit ID = [timestamp | datacenter ID | machine ID | sequence number]. Roughly time-sortable, unique, compact — industry standard, reused inside almost every case study below (order IDs, message IDs, tweet IDs).

## 2.16 Distributed Locks & Coordination

- Used to ensure only one process/node performs a critical action at a time (e.g. seat booking, inventory decrement).
- **Redis-based lock (Redlock):** SET key value NX PX ttl — simple, fast, good enough for most interviews.
- **Zookeeper / etcd:** strongly consistent coordination service — used for leader election, config management, distributed locks where correctness > speed.
- Always mention lock TTL / expiry to avoid deadlock if a node crashes while holding the lock.

## 2.17 Data Partitioning for Analytics/Search

- Denormalization: duplicate data across tables/documents to avoid expensive joins at read time — common in NoSQL feed/timeline design.
- CQRS (Command Query Responsibility Segregation): separate write model (normalized, transactional) from read model (denormalized, optimized for fast reads) — read model rebuilt async via events.
- Materialized views / pre-computation: compute expensive aggregates (trending items, feed ranking) ahead of time via batch/stream jobs instead of at request time.

## 2.18 Security Basics Worth Mentioning

- AuthN vs AuthZ: Authentication (who are you — OAuth2/JWT) vs Authorization (what can you do — RBAC/ABAC).
- JWT: stateless, self-contained token — good for scaling auth across microservices without a shared session store.
- TLS everywhere, encrypt PII at rest, and for payments always mention PCI-DSS compliance and tokenization (never store raw card numbers).

## Part 3: Reusable Building Blocks

---

These mini-systems show up as SUB-COMPONENTS inside almost every big case study in Part 4. Learn them once as standalone designs, then just "plug them in" during the real interview.

---

### 3.1 Rate Limiter (as a service)

- Requirement: limit each client/API-key to N requests per time window; return HTTP 429 when exceeded.
- Approach: Token Bucket algorithm, counters stored in Redis (in-memory, fast, supports atomic INCR + EXPIRE), one counter key per (client\_id, window).
- Deploy the limiter at the API Gateway layer so it protects all downstream services centrally.
- Distributed edge case: multiple gateway instances need a SHARED counter store (Redis), not per-instance local counters, else limits get multiplied by instance count.

### 3.2 Unique ID Generator Service

- Use Snowflake-style: [41 bits timestamp][10 bits machine/worker ID][12 bits sequence] = 63 bits, fits in a signed 64-bit int.
- Machine ID assigned via Zookeeper/config at startup so no two workers collide.
- Roughly time-sortable — useful for pagination (e.g. "load older messages") without needing a separate timestamp index.

### 3.3 URL Shortener (classic warm-up question)

- Core flow: POST long\_url → generate short code (base62 encode of an auto-incrementing/Snowflake ID, ~7 chars covers  $62^7 \approx 3.5$  trillion URLs) → store mapping → return short URL.
- Read path (GET /{code}): look up in cache (Redis) first — reads vastly outnumber writes (100:1+) — fall back to DB on miss, then 301/302 redirect.
- DB: simple key-value store is enough (short\_code → long\_url, created\_at, expiry, owner). Shard by hash(short\_code) if scale demands it.
- Extra: click analytics via async event to a queue → batch-processed into an analytics store, so it never slows down the redirect path.

### 3.4 Distributed Cache Layer

- Redis Cluster: data sharded across nodes via consistent hashing (hash slots, 16384 slots in real Redis Cluster).
- Replication per shard (1 primary + 1–2 replicas) for read scaling and failover.
- Client-side or proxy-based routing (e.g. Twemproxy) to know which node owns which key.

### 3.5 Notification / Alerting System (push, SMS, email)

- Ingestion: producers (order-service, chat-service) publish a `NotificationRequested` event to a queue (Kafka) — fully decoupled from the notification system itself.
- Notification service consumes events, applies user preferences (do-not-disturb, channel opt-in) and templates the message.

- Fan-out to channel-specific workers: Push (FCM/APNs), SMS (via telecom gateway/Twilio-like provider), Email (SES-like).
- Retry with exponential backoff + dead-letter-queue for failed sends; store delivery status for read receipts.
- For real-time in-app updates (order status, chat) use WebSockets or long-polling instead of push notifications for lower latency.

### 3.6 Search Autocomplete / Typeahead

- Data structure: Trie (prefix tree) holding top-K popular queries/results per prefix, refreshed periodically (not on every keystroke request) via an offline batch job over search logs.
- Trie kept in-memory across sharded servers (shard by first 1–2 characters of prefix) since it must serve in single-digit milliseconds.
- Client-side debouncing (wait ~150–300ms after typing stops) to cut down request volume before it even hits the server.

### 3.7 Distributed Job Scheduler / Cron

- Use case: "send reminder 30 min before movie show", "auto-cancel unpaid order after 10 min".
- Approach: jobs stored in DB with a `run\_at` timestamp; a pool of worker nodes poll a priority queue (or use a delayed-message feature of Kafka/SQS) for due jobs.
- Use a distributed lock (Section 2.16) so only ONE worker picks up and executes a given job, even with multiple scheduler replicas running for HA.

## Part 4: Hot Case Studies

The most frequently asked HLD questions in Indian interviews right now (2025–26), based on recent rounds at Swiggy, Zomato, Ola, PhonePe, Amazon, Flipkart, and FAANG India. Each follows the RESHAPE framework from Part 1.

### 4.1 Design Zomato / Swiggy (Food Delivery Platform)

#### Functional Requirements

- Customer: search restaurants by location/cuisine, view menu, place order, track order live, pay online.
- Restaurant: receive order, accept/reject, mark ready-for-pickup.
- Delivery partner: get assigned nearby orders, navigate, update delivery status.
- Out of scope (state this explicitly): payment gateway internals, restaurant onboarding/KYC.

#### Non-Functional Requirements

- Availability > strict consistency for browsing/search (AP system) — but order + payment state must be strongly consistent (CP for that slice).
- Low latency search (<200ms) since it's the core engagement loop.
- High write throughput for live location pings from delivery partners (millions of GPS updates/sec at India scale).

#### Capacity Estimation (rough)

- ~50M DAU on peak platforms like Swiggy/Zomato combined — assume 5M orders/day → avg ~60 orders/sec, peak (lunch/dinner) ~5–10x → ~400–600 orders/sec at peak.
- Delivery partner GPS pings: ~500K active partners × 1 ping/4 sec ≈ 125K writes/sec — this is the real scale challenge, NOT the order flow.

#### High-Level Architecture

- Client Apps → API Gateway → microservices: Search Service, Order Service, Restaurant Service, Delivery/Dispatch Service, Payment Service, Notification Service.
- Search Service: restaurant + menu data indexed in Elasticsearch (geo-query + text search); results cached in Redis by (location-geohash, cuisine) for hot areas.
- Order Service: backed by SQL (Postgres/MySQL) — orders need ACID transactions (inventory/menu availability decrement, payment linkage).
- Dispatch/Matching Service: assigns nearest available delivery partner using geospatial index (geohash or Google's S2/H3 cells) + a scoring function (distance, partner rating, current load).
- Location Service: ingest high-frequency GPS pings via Kafka → written to a fast KV/wide-column store (Cassandra) or in-memory geospatial store (Redis GEO commands) for "nearest partner" queries; NOT written to the primary SQL DB.
- Real-time order tracking pushed to customer app via WebSocket/long-poll, backed by pub-sub (Redis Pub/Sub or Kafka) so any app server instance can push updates regardless of which one the client is connected to.

#### Data Model (simplified)

| Entity           | Key Fields                                                                                 |
|------------------|--------------------------------------------------------------------------------------------|
| Restaurant       | id, name, geohash, cuisine[], menu_id, rating, is_open                                     |
| Order            | id (Snowflake), customer_id, restaurant_id, items[], status, total, payment_id, created_at |
| DeliveryPartner  | id, current_geohash, status(available/busy/offline), rating                                |
| OrderStatusEvent | order_id, status, timestamp (append-only, event-sourced for tracking history)              |

## Deep Dive: Delivery Partner – Order Matching

- Divide the map into geohash cells (or Uber's H3 hexagons); index available partners per cell in Redis (GEO commands give  $O(\log n)$  radius queries).
- On new order: query partners within expanding radius (start 1km → 3km → 5km) until enough candidates found; score by distance + idle time + acceptance rate; send push notification to top-K candidates; first to accept wins (optimistic) or auto-assign to best match.
- Handle race conditions with a short-TTL distributed lock per order during the assignment window so two partners can't both accept the same order.

## Trade-offs / What Interviewers Probe

- "What if the restaurant marks an item unavailable mid-order?" → answer: optimistic locking / version check on menu item at order-confirm time, fallback to partial refund flow.
- "How do you scale search during a flash sale/lunch rush?" → pre-warmed cache for popular areas, read replicas for Elasticsearch, autoscaling app tier behind LB.

### ▮ India-Specific Angle

- Interviewers at Swiggy/Zomato love asking about GPS ping scale and battery/network optimization (adaptive ping frequency based on movement) — mention this explicitly.
- Also expect a follow-up on surge/dynamic delivery fee pricing during peak hours — same pattern as Uber surge pricing (see 4.2).

## 4.2 Design Uber / Ola (Ride-Hailing Platform)

### Functional Requirements

- Rider: request ride (pickup + drop location), see ETA + fare estimate, track driver, pay, rate.
- Driver: go online/offline, receive ride requests, navigate, complete trip.
- Out of scope: driver onboarding/document verification, in-app payments internals.

### Non-Functional Requirements

- Extremely low-latency matching (<a few seconds) — core UX metric.
- High availability for location updates; eventual consistency acceptable for driver location (few-second staleness fine).
- Strong consistency required for trip state transitions (requested → accepted → ongoing → completed) to avoid double-booking a driver.

### Capacity Estimation

- ~5M active drivers in India across platforms, pinging location every 4 sec → ~1.25M location writes/sec at peak — this dominates system design, same as Swiggy/Zomato dispatch.
- Ride requests: assume 10M rides/day → ~115 QPS average, ~500+ QPS at peak (office hours, rain — classic Indian surge trigger).

### High-Level Architecture

- Core services: Location Service, Matching/Dispatch Service, Trip Service, Pricing Service, Payment Service, Notification Service — all behind an API Gateway.
- Location Service: same geohash/H3-cell pattern as food delivery — in-memory geospatial index (Redis GEO or custom quad-tree) sharded by region, refreshed every few seconds per driver.
- Matching Service: on ride request, query nearby available drivers in expanding radius; score by ETA (not just straight-line distance — use a routing/ETA microservice backed by map data); reserve driver with short-lived lock, send request, driver has ~10-15 sec to accept before reassigning.
- Trip Service: source of truth for trip state machine, backed by SQL for strong consistency on state transitions (a driver can only be in ONE active trip at a time — enforce via unique constraint / conditional update).
- Pricing Service: computes fare estimate combining base fare + distance + time + surge multiplier.

### Deep Dive: Surge Pricing

- Real-time supply/demand ratio computed per geo-cell:  $\text{surge\_multiplier} = f(\text{open\_ride\_requests} / \text{available\_drivers})$  in that cell, recalculated every 30–60 sec via a streaming job (Kafka Streams/Flink) consuming ride-request and driver-availability events.
- Multiplier cached in Redis per geo-cell so the pricing service reads it in  $O(1)$  without recomputing per request.
- Trade-off to mention: too-frequent surge updates cause rider confusion/complaints (a well-known real-world issue) — smoothing/capping the rate of surge change is a legitimate design discussion point.

### Deep Dive: Preventing Double-Booking a Driver

- Use a conditional/optimistic update: `UPDATE drivers SET status='assigned' WHERE id=? AND status='available'` — if 0 rows affected, someone else grabbed the driver first; retry matching with next candidate.
- Alternative: short-TTL distributed lock (Redis SETNX) per driver during the offer window — simpler to reason about, releases automatically if the offer processing crashes.

### Trade-offs / What Interviewers Probe

- "Driver goes offline mid-trip (network drop) — what happens?" → heartbeat/timeout mechanism; if no location ping for X sec, flag trip for manual/ops review, don't auto-cancel immediately (network flakiness is common on Indian mobile networks).

- "How do you estimate ETA accurately?" → mention a dedicated routing engine (like OSRM/Google Maps API) using road-graph + real-time traffic data, not straight-line distance.

#### ▮ India-Specific Angle

- Be ready to discuss 2-wheeler/auto-rickshaw specific matching (shorter radius, different pricing) — a common follow-up at Ola/Rapido interviews.
- Interviewers often ask you to compare Uber's H3 hexagonal grid system vs simple geohashing — know that H3 gives more uniform cell shapes/neighbor queries.

## 4.3 Design WhatsApp (Real-Time Messaging)

### Functional Requirements

- 1:1 chat and group chat, message delivery status (sent/delivered/read), online/last-seen presence, media sharing.
- Out of scope: end-to-end encryption key exchange internals (mention it exists via Signal Protocol, don't derive it).

### Non-Functional Requirements

- Very low latency (<100ms) for message delivery.
- High availability, AP-leaning — a message should never be silently lost, but brief delivery delay is acceptable.
- Massive scale: billions of messages/day, mostly small text payloads.

### Capacity Estimation

- ~500M+ India users, ~50 messages/user/day → 25B messages/day → ~290K messages/sec average, ~1M+/sec at peak.
- Storage: avg message ~100 bytes × 25B/day ≈ 2.5 TB/day of pure message data (before media).

### High-Level Architecture

- Clients maintain a persistent connection (WebSocket or custom TCP protocol like MQTT/XMPP) to a Chat/Connection Gateway server — NOT plain HTTP request/response, since server must PUSH messages.
- Connection Gateway servers are stateful (hold the live socket) — a Session/Presence Service maps user\_id → which gateway server + connection they're on (stored in Redis, since it changes often and needs fast lookup).
- Message flow: Sender → their Gateway → Message Service (persists message, assigns Snowflake ID) → looks up recipient's gateway via Session Service → forwards to that gateway → pushed to recipient if online, else queued.
- Offline delivery: message queued (per-user queue, e.g. in Kafka partitioned by user\_id, or a dedicated store) and delivered on next connect; push notification (FCM/APNs) sent to wake the app.
- Message storage: wide-column store (Cassandra) partitioned by chat\_id/conversation\_id, clustered by timestamp — optimized for "give me last N messages in this chat" access pattern, not for cross-chat queries.

### Deep Dive: Delivery / Read Receipts (Single, Double, Blue Tick)

- Sent tick: written once Message Service persists + acks the message.
- Delivered tick: recipient's client sends an ACK event back up through its gateway once the message reaches the device — propagated back to sender's gateway and pushed to sender.
- Read tick: client sends a read-receipt event when the chat is opened, same propagation path, batched (not per-message) to reduce chatter for group chats.

### Deep Dive: Group Chat Fan-out

- On send, message is written once to storage; then fanned out to each group member's queue/gateway lookup — for large groups (256 max on WhatsApp) this is a simple loop, not a big-fan-out problem like a celebrity's Twitter feed.
- Group metadata (member list) cached aggressively since it's read on every single message send.

### Trade-offs / What Interviewers Probe

- "How do you know which gateway server a user is connected to, across thousands of gateway instances?" → the Session Service / connection registry in Redis is THE key insight — always mention it explicitly.
- "What if the gateway server holding a user's socket crashes?" → client detects disconnect, reconnects (with backoff) to a new gateway instance (chosen by LB), re-registers session; any undelivered messages queued in that window are delivered on reconnect.

### 🚩 Interview Tip

- The #1 differentiator vs a generic "chat app" answer: explicitly say messaging needs STATEFUL, LONG-LIVED connections (WebSocket) with a separate presence/session directory — many candidates default to HTTP polling and lose marks here.

## 4.4 Design Instagram (Photo/Video Feed + Stories)

### Functional Requirements

- Upload photo/video with caption, follow/unfollow users, view a personalized home feed, view stories (24hr expiry), like/comment.
- Out of scope: recommendation ML ranking model internals, ad insertion.

### Non-Functional Requirements

- Extremely read-heavy (feed viewed constantly vs posted occasionally) — heavy caching + read replicas.
- Feed can be eventually consistent (a few seconds/minutes delay showing a new post is fine).
- High availability for media delivery — CDN is mandatory, not optional.

### Capacity Estimation

- ~200M India users, ~20% post daily → 40M posts/day → ~460 writes/sec average; feed reads are ~100–200x that — tens of thousands of reads/sec.
- Media storage: assume avg 2MB/photo × 40M/day ≈ 80 TB/day raw — must go to object storage (S3-like), never a DB.

### High-Level Architecture

- Media Upload: client uploads directly to object storage (S3-compatible) via a pre-signed URL (server never proxies large binary through itself); triggers async processing (thumbnail generation, transcoding for video, virus/content-moderation scan) via a queue.
- Metadata (post\_id, user\_id, caption, media\_url, timestamp, like\_count) stored separately in a fast DB (Cassandra or sharded MySQL).
- CDN in front of object storage for actual image/video bytes — this is what makes global image delivery fast.

### Deep Dive: News Feed Generation — Fan-out on Write vs Fan-out on Read

| Approach                  | How It Works                                                                                        | Best For                                                                                                          |
|---------------------------|-----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Fan-out on Write (Push)   | When user posts, immediately push post_id into EVERY follower's precomputed feed list (cache/inbox) | Fast feed reads (just fetch precomputed list). Bad for celebrities with 50M followers — huge write amplification. |
| Fan-out on Read (Pull)    | Feed is computed at read-time by merging recent posts from all people you follow                    | No write amplification. Slower reads, must merge/sort from many sources at request time.                          |
| Hybrid (used in practice) | Push for normal users; celebrities/high-follower accounts use pull, merged in at read time          | Best of both — THIS is the answer interviewers want.                                                              |

- Precomputed feed lists stored in Redis (a sorted set per user, scored by timestamp) for fast fan-out-on-write reads.

### Deep Dive: Stories (24-hour expiry)

- Store with a TTL directly (Redis TTL, or a DB row + scheduled cleanup job) — natural fit for ephemeral data, avoids manual deletion logic.
- Separate, smaller "story ring" feed computed similarly to the main feed but scoped to just the last 24h window.

### Trade-offs / What Interviewers Probe

- "What happens when a celebrity with 50M followers posts?" → THIS is the question that tests if you know fan-out trade-offs — always bring up the hybrid approach proactively.
- "How do you rank the feed (not just chronological)?" → acceptable to say: a separate ranking/ML service scores candidate posts (engagement prediction) — don't over-engineer the ML if not asked, just show awareness.

🚩 **Interview Tip**

- Fan-out on write vs read is THE single most-tested deep dive across Instagram, Twitter/X, and Facebook-style feed questions. Memorize the hybrid answer cold.

## 4.5 Design Twitter / X (Feed + Search + Trends)

### Functional Requirements

- Post a tweet (280 chars + media), follow users, view home timeline, search tweets, view trending topics.
- Out of scope: ad ranking, DM/messaging (covered by WhatsApp case study pattern).

### Non-Functional Requirements

- Read:write ratio extremely skewed (100:1+) — read path must be near-instant.
- Eventual consistency acceptable for timeline; search index can lag by seconds.

### High-Level Architecture

- Same fan-out-on-write/read hybrid as Instagram (Section 4.4) for the home timeline — don't re-derive, just reuse and say so explicitly in the interview to save time.
- Tweet data stored in a wide-column store (Cassandra) keyed by tweet\_id (Snowflake), with a secondary index/table for user\_id → tweet\_ids (their own timeline).
- Search: tweets asynchronously indexed into Elasticsearch via a change-data-capture stream (Kafka) off the primary write path — keeps write path fast, search is near-real-time not real-time.

### Deep Dive: Trending Topics

- Streaming aggregation job (Kafka Streams/Flink/Spark Streaming) counts hashtag/keyword occurrences in a sliding time window (e.g. last 15–60 min), maintains a running Top-K (via a min-heap or Count-Min Sketch for approximate counting at scale).
- Count-Min Sketch is the key 'senior' answer here: exact counting of every word globally is too memory-expensive, so an approximate probabilistic data structure trades a small, bounded error for huge memory savings.
- Top-K refreshed every N seconds and cached — trends don't need to update every millisecond.

#### ▮ Interview Tip

- Mentioning Count-Min Sketch or HyperLogLog (for unique-view-count style problems) is a strong signal of depth — use them specifically for the 'trending' or 'unique visitor count' deep dives.

## 4.6 Design Netflix / Hotstar (Video Streaming)

### Functional Requirements

- Browse catalog, search titles, stream video adaptively based on network quality, resume playback, personalized recommendations.
- Hotstar-specific twist (very commonly asked): LIVE streaming for cricket/sports with tens of millions of concurrent viewers.

### Non-Functional Requirements

- Massive bandwidth — CDN is the entire ballgame, not an afterthought.
- Low startup latency and no buffering — adaptive bitrate streaming is mandatory.
- For live sports: near-zero tolerance for downtime during peak moments (a wicket falling), extreme concurrent-viewer spikes.

### High-Level Architecture (Video-on-Demand)

- Upload pipeline: raw video → Transcoding Service converts into multiple resolutions/bitrates (240p to 4K) and chunks it into small segments (HLS/DASH protocols) — this is done ONCE, asynchronously, offline.
- Encoded chunks pushed to object storage, then replicated to CDN edge servers worldwide/across India ISPs.
- Client player uses Adaptive Bitrate Streaming (ABR): monitors its own download speed/buffer health and requests the next chunk at a higher or lower quality accordingly — THIS is the single most important concept to explain clearly.
- Metadata service (catalog, search, recommendations) is a completely separate, much smaller-scale system from the video-byte-serving path — call this out explicitly, candidates often conflate the two.

### Deep Dive: Live Streaming at Massive Concurrent Scale (Hotstar Cricket)

- Live video is also chunked (short segments, e.g. 2–6 sec) and pushed to CDN continuously — clients poll/ fetch a manifest file listing available chunks, same ABR logic applies.
- Multi-CDN strategy: don't rely on a single CDN provider — use several (and even ISP-level edge caching within India, since a huge % of traffic is India-only during IPL/World Cup) to avoid one provider's capacity becoming the bottleneck.
- Pre-scale infrastructure ahead of known high-traffic events (match start times are predictable) rather than relying purely on reactive autoscaling, which is too slow for the traffic cliff at a match's start.
- Degrade gracefully under extreme load: serve lower default bitrate to new joiners during a spike, queue/ throttle non-critical requests (comments, live-poll features) to protect the core video path.

### Trade-offs / What Interviewers Probe

- "Why not just always serve the highest quality?" → bandwidth cost + buffering risk on India's variable mobile networks — ABR exists precisely for this.
- "How is live different from VOD technically?" → no ability to pre-transcode everything ahead of time; encoding must happen in near-real-time with a small delay (the 'broadcast delay'), and CDN cache TTLs must be very short since content is constantly new.

#### ► India-Specific Angle

- Hotstar's real 2019 World Cup final scaling story (25M+ concurrent viewers) is a favorite reference point in Indian interviews — knowing the multi-CDN + pre-scaling + graceful-degradation narrative above covers that discussion well.

## 4.7 Design BookMyShow / Ticketmaster (Concurrent Seat Booking)

### Functional Requirements

- Browse shows/events, view seat map, select seats, hold seats temporarily, pay, confirm booking.
- Out of scope: payment gateway internals, dynamic pricing for premium seats (mention it exists, don't derive).

### Non-Functional Requirements

- Strong consistency is NON-NEGOTIABLE for seat selection — two people must never be sold the same seat. This is a CP problem, unlike most consumer apps.
- Must handle massive concurrent demand spikes (blockbuster movie/concert ticket drop) — thousands of users hitting the same show simultaneously.

### High-Level Architecture

- Show/Seat Service is backed by a relational DB (Postgres/MySQL) — this is one of the few consumer-scale systems where you should NOT default to NoSQL, because the core problem is transactional correctness under contention.
- Waiting room / virtual queue in front of high-demand shows: incoming users are queued (via Redis-backed token system) and let into the booking flow at a controlled rate — protects the DB from a thundering herd on ticket-drop moments.

### Deep Dive: Preventing Double-Booking of a Seat

| Approach                 | How It Works                                                                                      | Note                                                                                                                    |
|--------------------------|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Pessimistic Locking      | SELECT ... FOR UPDATE on the seat row when a user selects it                                      | Simple, guarantees correctness, but can bottleneck under very high contention on the same row                           |
| Optimistic Locking       | UPDATE seats SET status='held', version=version+1 WHERE id=? AND status='available' AND version=? | No lock held during user think-time; if 0 rows affected, someone else got it — retry/show error. Preferred in practice. |
| Distributed Lock (Redis) | SETNX seat_lock:{id} with short TTL during the hold window                                        | Works well combined with DB optimistic check as a fast first-pass filter                                                |

- **Seat hold with TTL:** once a user selects seats, mark them 'held' for a fixed window (e.g. 5–10 min) via a TTL (Redis expiry or a scheduled job) — if payment isn't completed in time, seats auto-release back to 'available'.
- This TTL-based hold is THE key idea interviewers are fishing for — always state it explicitly and early in the deep dive.

### Trade-offs / What Interviewers Probe

- "How do you handle a flash sale where 100K users hit 'Book Now' in the same second for 500 seats?" → virtual queue + rate limiting at the edge, THEN optimistic locking on the actual seat rows — don't let raw traffic hit the transactional core directly.
- "Why SQL here, when most modern systems are told to prefer NoSQL?" → because the core invariant ("exactly one buyer per seat") is a textbook ACID transaction problem — correctly identifying WHEN to use SQL over NoSQL is exactly what this question tests.

## 4.8 Design UPI / Payment Gateway (PhonePe, Paytm, Razorpay style)

Extremely hot in Indian interviews given the fintech boom (PhonePe, Razorpay, Cred, Paytm, Google Pay India R&D centers all hire heavily for this).

### Functional Requirements

- Initiate a payment/transfer between two bank accounts via a Virtual Payment Address (VPA), check balance, view transaction history, handle refunds.
- Out of scope: actual bank core-banking integration internals (mention it as an external dependency, NPCI-style switch).

### Non-Functional Requirements

- Strong consistency and durability are ABSOLUTE — money must never be double-debited, never lost, always reconcilable. This is the strictest CP + ACID requirement of any case study here.
- Idempotency is mandatory (Section 2.12) — network retries on flaky mobile connections must never cause a double debit.
- High availability still matters (a payment system that's 'down' is a business disaster) — the real skill is achieving BOTH strong consistency and high availability, which is why real payment systems invest heavily in infra (this tension is worth naming explicitly).

### High-Level Architecture

- Client App → API Gateway → Payment Orchestrator Service → talks to Bank Adapter services (one per bank / via NPCI switch) → core Ledger Service.
- Ledger Service: the source of truth for balances/transactions, backed by a relational DB with strict ACID transactions — this is the one place in the whole guide where you should NOT casually suggest NoSQL or 'eventual consistency is fine'.
- Every transaction follows a two-phase-ish flow: DEBIT from payer (reserve funds) → CREDIT to payee → mark transaction COMPLETE, with each step logged as an immutable ledger entry (append-only, never update-in-place) so the full history is always auditable and reconstructable.

### Deep Dive: Idempotency & Exactly-Once Semantics

- Client generates a unique Idempotency-Key (UUID) per payment attempt BEFORE sending the request; if the app retries due to a timeout, it resends the SAME key.
- Server checks a fast lookup (Redis or a dedicated idempotency table) for that key: if seen before, return the STORED result instead of reprocessing — never re-execute the debit.
- This solves the classic "payment succeeded on the backend but the app showed a timeout/error, so the user tries again" scenario — a real, extremely common failure mode on Indian mobile networks.

### Deep Dive: Distributed Transaction Across Two Bank Accounts (Debit + Credit)

- Because debit and credit may hit DIFFERENT bank systems (different DBs, possibly different companies entirely via NPCI), a single ACID transaction spanning both isn't possible — this is where the Saga pattern applies.
- **Saga pattern:** a sequence of local transactions, each with a corresponding COMPENSATING transaction. If step 2 (credit) fails after step 1 (debit) succeeded, an automatic REVERSAL/refund compensating transaction runs to undo the debit.
- Every transaction moves through an explicit state machine (INITIATED → DEBITED → CREDITED → SETTLED, or → FAILED → REVERSED) persisted durably, so a crashed orchestrator can resume/reconcile exactly where it left off by reading the last known state.
- A separate async Reconciliation Service periodically compares ledger entries against bank statements/ NPCI settlement files to catch and auto-resolve any edge-case mismatches — mentioning this shows real-world payments maturity.

### Trade-offs / What Interviewers Probe

- "What if the credit step fails after debit succeeded?" → THE core question of this entire case study — answer with Saga + compensating reversal transaction, not "it just won't fail".
- "Why not a single 2-Phase-Commit (2PC) transaction across both banks?" → 2PC requires a coordinator with locking control over BOTH participants and blocks if the coordinator or a participant crashes mid-commit — infeasible across independent bank systems/companies; Sagas trade strict atomicity for availability + are the industry-standard answer here.

#### ▮ India-Specific Angle

- Know the words 'NPCI', 'VPA', 'Saga pattern', and 'idempotency key' cold — dropping these terms naturally (not just as buzzwords, but tied to the failure scenario they solve) is what separates a strong fintech-interview answer from a generic one.

## 4.9 Design Google Drive / Dropbox (File Storage & Sync)

### Functional Requirements

- Upload/download files and folders, sync across multiple devices, share files with permissions, version history.
- Out of scope: real-time collaborative editing (like Google Docs) — that's a separate, harder OT/CRDT problem, mention it exists as a follow-up extension.

### Non-Functional Requirements

- Storage efficiency matters (dedupe identical file blocks across users).
- Sync should be fast and low-bandwidth (only transfer CHANGED parts of a file, not the whole file every time).

### High-Level Architecture

- Files are split into fixed-size or content-defined chunks (blocks) on the client BEFORE upload; each block is hashed (e.g. SHA-256).
- Client asks the Metadata Service which of these block-hashes already exist in storage; only NEW/changed blocks are actually uploaded — this gives both deduplication (across all users, if content matches) and delta-sync (only upload what changed in a file edit).
- Blocks stored in object storage (S3-like), Metadata Service (SQL or document DB) maps file → ordered list of block hashes — reconstructing a file is just concatenating its blocks in order.
- Sync Service pushes change notifications to other logged-in devices of the same user (via a long-lived connection or a lightweight notify-then-pull model) so they know to fetch the new blocks.

### Deep Dive: Handling Sync Conflicts

- If the same file is edited offline on two devices, on reconnect the system detects a version conflict (via a version number or last-modified timestamp mismatch) and either auto-merges (rare, risky) or creates a 'conflicted copy' file for the user to manually resolve — this is the pragmatic real-world answer (this IS what Dropbox actually does).

### Trade-offs / What Interviewers Probe

- "How do you avoid re-uploading a huge file after a tiny edit?" → chunking + hash-based delta sync is the entire answer — state it clearly and early.

## 4.10 Design an E-Commerce Platform (Flipkart / Amazon India style)

### Functional Requirements

- Browse/search products, add to cart, place order (with inventory check), track order, handle returns.
- Out of scope: seller onboarding, ads/sponsored listings, recommendation engine internals.

### Non-Functional Requirements

- Search/browse can be eventually consistent (AP) — seeing a product that JUST sold out is an acceptable, rare edge case.
- Inventory decrement + order placement must be strongly consistent (CP) — must never oversell a limited-stock item.
- Massive read:write skew (browsing >> buying), plus predictable extreme spikes (Big Billion Days / Great Indian Festival sale).

### High-Level Architecture

- Catalog/Search Service: product data in Elasticsearch for full-text + filter search, heavily cached (Redis/CDN for images), read-replica heavy.
- Cart Service: can be a simple key-value store (Redis) per user — cart contents don't need ACID until checkout.
- Order + Inventory Service: relational DB, since "decrement stock AND create order" must be one atomic transaction — use optimistic locking (version column) on the inventory row, same pattern as the BookMyShow seat-hold problem (Section 4.7).

### Deep Dive: Flash Sale / Big Billion Days Scale Problem

- Same core pattern as BookMyShow: a virtual waiting-room/queue in front of the checkout flow for hyper-limited-stock items, so the inventory DB never sees more concurrent writers than it can actually handle.
- Pre-reserve inventory in a fast store (Redis DECR, atomic) as the first gate; only on success does the request proceed to create the durable order record in SQL — this keeps the hot path fast while keeping the SQL DB as the eventual source of truth.
- Static asset delivery (product images, category pages) pushed hard to CDN + aggressive caching, since app-tier and DB capacity is the truly scarce resource during a sale, not bandwidth.

### Trade-offs / What Interviewers Probe

- "1000 people click 'Buy' on the last 10 units — what happens?" → atomic Redis DECR as a fast gate + optimistic-locked SQL transaction as the source of truth; the 990 who lose the race get an immediate, clear 'out of stock' response rather than a slow, confusing wait.

#### ► Pattern Recognition (Very Important)

- Notice BookMyShow (4.7), UPI Payments (4.8), and E-commerce (4.10) all reduce to the SAME core problem: 'many concurrent actors racing for a scarce, strongly-consistent resource.' The answer is always some combination of: virtual queue → fast atomic pre-check (Redis) → optimistic-locked SQL transaction → TTL-based hold/compensation on failure.
- Recognizing and naming this pattern OUT LOUD in an interview ("this is the same class of problem as seat booking") is a very strong signal of maturity — it shows you understand principles, not memorized answers.

## Part 5: India-Specific Interview Patterns

What each category of company actually probes for, based on recent (2025–26) interview patterns.

| Company Type                                               | What They Probe For                                                                                                                                                                                       |
|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Product unicorns (Swiggy, Zomato, Ola, Meesho)             | Geo-spatial matching at scale, real-time tracking, handling India-scale traffic spikes (festival sales, cricket matches), cost-consciousness (India margins are thin — mention infra cost trade-offs).    |
| Fintech (PhonePe, Razorpay, Cred, Paytm)                   | Strong consistency, idempotency, Saga/compensating transactions, fraud detection at a high level, regulatory awareness (RBI guidelines, PCI-DSS) — correctness over cleverness.                           |
| FAANG India R&D centers                                    | Classic distributed-systems fundamentals (Part 2) applied rigorously; expect deep follow-ups on ONE component rather than a broad tour; strong signal wanted on trade-off articulation.                   |
| Service-based (TCS Digital, Infosys, Cognizant, Capgemini) | More conceptual/breadth-first: SOLID + microservices vs monolith + basic HLD diagram + simple DB choice justification; less deep-dive pressure than product companies.                                    |
| Amazon (India offices)                                     | Leadership-principle-flavored follow-ups even in system design ('why did you choose that trade-off', 'what would you do differently at 10x scale') — be ready to justify every choice, not just state it. |

### Recently Trending Questions (2025–26)

- Design a UPI-style payment system (see 4.8) — the single hottest fintech question right now.
- Design a food/grocery delivery ETA + dispatch system (Zomato/Swiggy Instamart/Blinkit-style 10-minute delivery adds a new twist: much tighter SLA, smaller geo-radius, pre-positioned dark-store inventory).
- Design a ticket-booking waiting room / anti-scalper queue (BookMyShow / IRCTC Tatkal-style — IRCTC's Tatkal booking rush is a very commonly referenced real-world example in Indian interviews).
- Design a notification system that respects DND/opt-out preferences at India-scale (TRAI regulations on promotional SMS are sometimes brought up as a constraint).
- Design rate limiting / abuse prevention for an OTP service (extremely common at fintech/PhonePe-style interviews — combine with Section 2.13's token bucket).

#### General India-Context Tips

- Always factor in variable/flaky mobile network conditions (2G/3G pockets, intermittent connectivity) when discussing client-server design — this is a recurring theme interviewers expect for an India-market product.
- Cost-awareness is valued: mention when a cheaper option (e.g. SQS over Kafka for simple queueing, or a single-region deployment before multi-region) is a legitimate choice for an early-stage/cost-sensitive product.

## Part 6: One-Page Rapid Revision (Night Before The Interview)

### The Framework, In One Line

R-E-S-H-A-P-E: Requirements → Estimation → Storage/Data model → High-level design → API design → Pick a deep dive → Evaluate trade-offs.

### Decision Cheat Sheet

| If the question involves...                                    | Default to...                                                                  |
|----------------------------------------------------------------|--------------------------------------------------------------------------------|
| Money / balances / inventory count                             | SQL + ACID transaction + optimistic locking + idempotency key                  |
| Chat / feed / social data at huge scale                        | NoSQL (Cassandra/wide-column) + cache-aside + CDN for media                    |
| Two people racing for one scarce resource (seat, item, driver) | Virtual queue → atomic pre-check (Redis) → optimistic-locked DB txn → TTL hold |
| Real-time push (chat, order tracking, live scores)             | WebSocket / persistent connection + a session/presence directory service       |
| Search / autocomplete                                          | Elasticsearch (full text) or Trie (prefix, in-memory) depending on query type  |
| Huge read:write skew (feed, catalog, product pages)            | Aggressive caching (Redis) + read replicas + CDN for static assets             |
| Money moving across two independent systems (banks)            | Saga pattern with compensating transactions, not 2PC                           |
| Sudden massive spike (flash sale, cricket match, ticket drop)  | Pre-scaling + virtual queue + graceful degradation, not just autoscaling       |
| Need to generate IDs across many machines                      | Snowflake ID (time-sortable, no coordination needed)                           |
| Need to add/remove servers without remapping everything        | Consistent hashing                                                             |

### Numbers To Have Ready

- 1 day  $\approx 10^5$  seconds. Peak QPS  $\approx 2\text{--}3\times$  average QPS.
- Cache (Redis)  $\sim 1\text{ms}$ , same-DC network  $\sim 0.5\text{ms}$ , DB query  $\sim 5\text{--}10\text{ms}$ , cross-continent  $\sim 150\text{ms}$ .
- Always show your math out loud, even if approximate — the reasoning is graded, not decimal precision.

### The 5 NFRs To Always State Explicitly

- 1. Scale (DAU/QPS) 2. Consistency vs Availability (CAP) 3. Latency target 4. Durability requirement 5. Read:Write ratio.

### Phrases That Signal Seniority (use naturally, not as buzzword-dropping)

- "I'll prioritize Availability over Consistency here since a few seconds of stale data is an acceptable trade-off for this use case."
- "This is the same class of problem as seat-booking — scarce resource + concurrent writers — so I'll reuse that pattern."

- "I'd start with a modular monolith and split into microservices only where scaling or ownership needs diverge."
- "Let me state what's explicitly out of scope so we can go deep on what matters."

## Most Common Deep-Dive Questions Across ALL Case Studies

- How do you prevent double-booking/double-spending/overselling under high concurrency?
- How do you scale reads vs writes independently?
- What happens when a component (gateway server, driver's phone, a bank) fails mid-operation?
- How would this design change at 10x or 100x scale?
- Fan-out on write vs fan-out on read — which and why, for this specific feed/timeline?

### 📌 Final Reminder

- Think out loud. A mediocre design explained with clear reasoning beats a perfect design delivered silently.
- It's fine to say "I don't know the exact internals of X, but here's how I'd reason about it" — confidence in your PROCESS is what's actually being evaluated.
- Good luck, Shreya — you've got the CS fundamentals (crypto, networking, OS) already; this guide is just the vocabulary layer on top.